

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: Ch.Gouri Sai Sree

Roll No.: A24126552076

Date: 25-08-2026

1. TITLE

Develop a Student Profile Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To develop a mini web application using **JavaScript** that accepts student information through an HTML form, creates a **Student object using a JavaScript class**, and dynamically displays the submitted details using **DOM manipulation**.

3. OBJECTIVE

The objectives of this experiment are:

- To understand JavaScript classes and objects.
 - To create a Student class with properties such as Name, Roll Number, Department, and CGPA.
 - To create a Student object using user-provided values.
 - To understand DOM selection and manipulation.
 - To dynamically create and display HTML elements.
 - To implement event handling using a button click.
 - To apply CSS styling to the dynamically generated student profile.
-

4. THEORY

JavaScript is a programming language used to add dynamic and interactive functionality to web pages.

JavaScript Classes provide a blueprint for creating objects with properties and methods. In this application, a Student class is used to store student information such as name, roll number, department, and CGPA.

DOM (Document Object Model) represents the HTML document as a tree of objects, allowing JavaScript to access and modify webpage elements dynamically.

In this application:

- `class Student` is used to define the structure of a student object.
- `constructor()` initializes the student properties.
- `new Student()` creates a Student object using user-provided values.
- `document.getElementById()` is used to select HTML elements from the DOM.
- `.value` retrieves the values entered by the user.
- `addEventListener()` handles the button click event.
- `document.createElement()` dynamically creates new HTML elements.
- `textContent` adds student information to the dynamically created elements.
- `appendChild()` adds the created elements to the webpage.

The basic flow of the application is:

User Input → Button Click → Student Object → DOM Manipulation → Student Profile Display

5. ALGORITHM / PROCEDURE

1. Create an HTML file containing input fields for Name, Roll Number, Department, and CGPA.
 2. Add a **Create Profile** button to the webpage.
 3. Create a JavaScript file and define a Student class.
 4. Define the `constructor()` to initialize Name, Roll Number, Department, and CGPA.
 5. Select the Create Profile button using `document.getElementById()`.
 6. Add a click event listener to the button.
 7. Retrieve the values entered by the user from the input fields.
 8. Create a Student object using the new `Student()` constructor.
 9. Select the profile container from the DOM.
 10. Create HTML elements dynamically using `document.createElement()`.
 11. Display the student details using `textContent`.
 12. Add the dynamically created elements to the profile using `appendChild()`.
 13. Display the generated student profile on the webpage.
 14. Verify the output by entering different student details.
 15. Compare the actual output with the expected output.
-

6. PROGRAM

main.js

```
class Student {

    constructor(name, rollNumber, department, cgpa) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.department = department;
        this.cgpa = cgpa;
    }
}

const button = document.getElementById("displayBtn");

button.addEventListener("click", function () {

    const name = document.getElementById("name").value;
    const rollNumber = document.getElementById("rollNumber").value;
    const department = document.getElementById("department").value;
    const cgpa = document.getElementById("cgpa").value;

    const student = new Student(
        name,
        rollNumber,
        department,
        cgpa
    );

    const container = document.getElementById("profileContainer");

    container.innerHTML = "";

    const profile = document.createElement("div");
```

```
profile.className = "profile";

const heading = document.createElement("h2");

heading.textContent = "Student Profile";

profile.appendChild(heading);

const nameElement = document.createElement("p");
nameElement.textContent = "Name          : " + student.name;

const rollElement = document.createElement("p");
rollElement.textContent = "Roll No      : " + student.rollNumber;

const departmentElement = document.createElement("p");
departmentElement.textContent = "Department : " + student.department;

const cgpaElement = document.createElement("p");
cgpaElement.textContent = "CGPA          : " + student.cgpa;

profile.appendChild(nameElement);
profile.appendChild(rollElement);
profile.appendChild(departmentElement);
profile.appendChild(cgpaElement);

container.appendChild(profile);

});
```

main.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Profile</title>

<link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1>Student Profile</h1>

    <input type="text" id="name" placeholder="Enter Name">

    <input type="text" id="rollNumber" placeholder="Enter Roll
Number">

    <input type="text" id="department" placeholder="Enter Department">

    <input type="number" id="cgpa" placeholder="Enter CGPA"
step="0.01">

    <button id="displayBtn">Display Profile</button>

    <div id="profileContainer"></div>

  </div>

  <script src="profile.js"></script>

</body>
</html>
```

main.css

```
body {
  font-family: Arial, sans-serif;
```

```
        background-color: #f2f2f2;
        text-align: center;
        padding: 40px;
    }

    .container {
        width: 400px;
        margin: auto;
        background-color: white;
        padding: 25px;
        border-radius: 10px;
        box-shadow: 0 0 10px gray;
    }

    input {
        width: 90%;
        padding: 10px;
        margin: 8px;
        border: 1px solid #aaa;
        border-radius: 5px;
    }

    button {
        padding: 10px 20px;
        margin-top: 10px;
        background-color: #007bff;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    button:hover {
        background-color: #0056b3;
    }

    .profile {
        margin-top: 25px;
        padding: 20px;
        background-color: #e8f4ff;
```

```

border-radius: 8px;
border-left: 5px solid #007bff;
text-align: left;
}

.profile h2 {
text-align: center;
color: #007bff;
}

.profile p {
font-size: 16px;
margin: 10px 0;
}

```

7. CODE EXPLANATION

Code	Explanation
<code>class Student</code>	Defines a JavaScript class named <code>Student</code> to represent student information.
<code>constructor(name, rollNo, department, cgpa)</code>	Initializes the properties of a <code>Student</code> object.
<code>this.name = name</code>	Stores the student's name in the object.
<code>this.rollNo = rollNo</code>	Stores the student's roll number.
<code>this.department = department</code>	Stores the student's department.
<code>this.cgpa = cgpa</code>	Stores the student's CGPA.
<code>document.getElementById("createProfile")</code>	Selects the Create Profile button from the HTML document.
<code>addEventListener("click", function() {...})</code>	Executes the specified function when the button is clicked.

<code>document.getElementById("name").value</code>	Retrieves the name entered by the user.
<code>document.getElementById("rollNo").value</code>	Retrieves the roll number entered by the user.
<code>document.getElementById("department").value</code>	Retrieves the department entered by the user.
<code>document.getElementById("cgpa").value</code>	Retrieves the CGPA entered by the user.
<code>new Student(name, rollNo, department, cgpa)</code>	Creates a new Student object using the user-provided values.
<code>document.getElementById("profileContainer")</code>	Selects the container where the generated profile will be displayed.
<code>profileContainer.innerHTML = ""</code>	Clears any previously generated student profile.
<code>document.createElement("div")</code>	Dynamically creates a new <code><div></code> element.
<code>profile.className = "profile"</code>	Assigns the CSS class <code>profile</code> to the newly created element.
<code>document.createElement("h2")</code>	Dynamically creates a heading element for the profile title.
<code>document.createElement("p")</code>	Dynamically creates paragraph elements for student details.
<code>textContent</code>	Sets the text content of the dynamically created elements.
<code>appendChild()</code>	Adds the dynamically created elements to the profile.
<code>profileContainer.appendChild(profile)</code>	Adds the completed student profile to the webpage.

8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Actual Result
-----------	------	----------	------------	------	-----------------	---------------

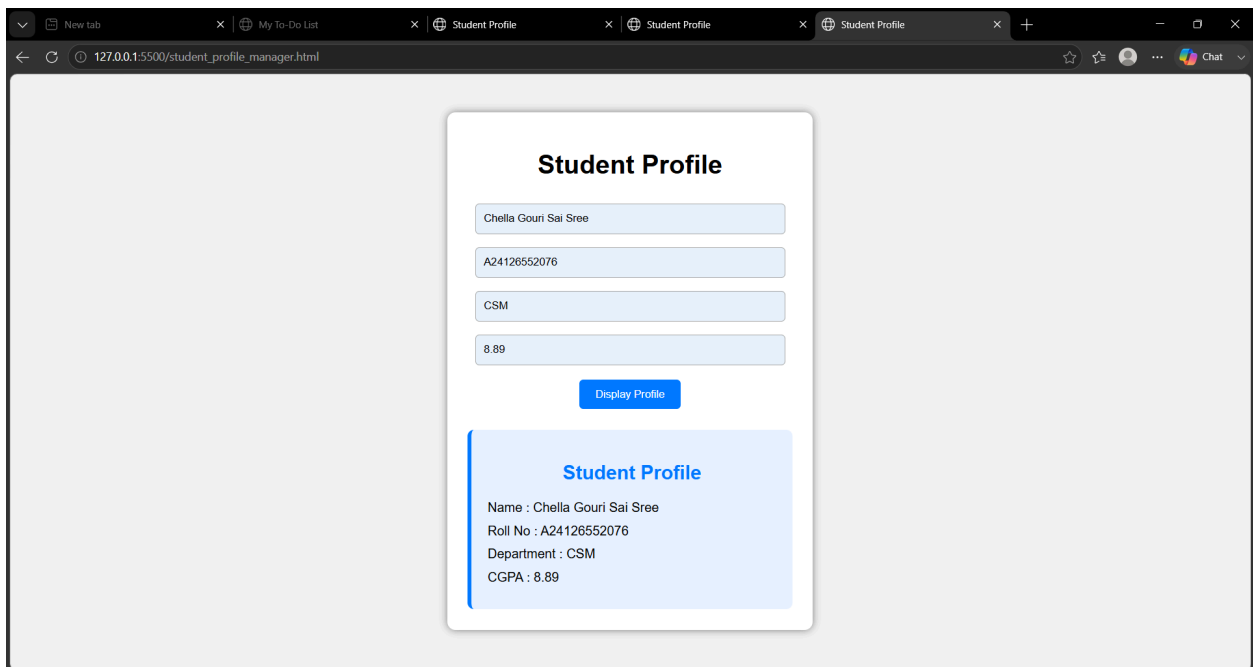
TC-01	chella gouri sai sree	241265520 76	CSM	8.89	Student profile displayed correctly	Student profile displayed correctly
TC-02	Bhuvan teja	241265520 84	CSM	9.20	Student profile displayed correctly	Student profile displayed correctly
TC-03	Lekhya sri	241265521 23	CSM	9.80	Student profile displayed correctly	Student profile displayed correctly
TC-04	sravan	241265520 85	CSM	9.90	Student profile displayed correctly	Student profile displayed correctly
TC-05	bhanu	241265520 68	CSM	10.00	Student profile displayed correctly	Student profile displayed correctly

Additional Validation Test

Test Case	Input	Expected Result	Actual Result
TC-05	Enter a new set of student details after generating a profile	Previous profile should be cleared and the new profile should be displayed	Previous profile cleared and new profile displayed correctly

9. OUTPUT

Output : code execution



The screenshot shows a web browser with multiple tabs. The active tab is titled 'Student Profile' and displays a web page with the URL '127.0.0.1:5500/student_profile_manager.html'. The page features a 'Student Profile' form with four input fields containing the text 'Chella Gouri Sai Sree', 'A24126552076', 'CSM', and '8.89'. Below these fields is a blue 'Display Profile' button. Below the button, the generated profile is displayed in a light blue box with the title 'Student Profile' and the following details: Name : Chella Gouri Sai Sree, Roll No : A24126552076, Department : CSM, and CGPA : 8.89.

10. RESULT

Thus, a **Student Profile Generator** was successfully developed using **JavaScript classes and DOM manipulation**. A Student class was created, user-provided student details were used to create objects, and the profile was dynamically generated and displayed on the webpage. The application was successfully verified using multiple test cases.
